

Ankara Üniversitesi
Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü

IoT Tabanlı Akıllı Şehir Sıcaklık İzleme Sistemi
(BLM3522 - Bulut Bilişim ve Uygulamaları)

Ders Öğretim Üyesi
Öğretim Görevlisi Enver BAĞCI



Github Proje Linki ;

<https://github.com/muhammetsah1n/BLM3522-Proje3>

“Video linki github’da mevcuttur”

Hazırlayan
Muhammet Şahin - 23291284

Ankara-2026

İÇİNDEKİLER

1. Giriş
2. Proje Tanımı
3. Kullanılan Teknolojiler
4. Sistem Mimarisini
5. Veri Akış Yapısı
 - 5.1 MQTT Veri Gönderimi
 - 5.2 Lambda Veri İşleme Süreci
 - 5.3 DynamoDB Veri Saklama Yapısı
6. Sensör Simülasyonu
 - 6.1 Python Sensör Yapısı
 - 6.2 Rastgele Veri Üretimi
 - 6.3 MQTT Publish İşlemleri
7. Dashboard Geliştirme
 - 7.1 Streamlit Yapısı
 - 7.2 Grafik Oluşturma
 - 7.3 Gerçek Zamanlı Güncelleme
 - 7.4 Alarm Sistemi
8. AWS Üzerinde Kurulum
 - 8.1 IoT Thing Oluşturma
 - 8.2 Sertifika Yönetimi
 - 8.3 IoT Rule Oluşturma
 - 8.4 Lambda Yetkilendirme
 - 8.5 DynamoDB Kurulumu
9. Sonuç

1. Giriş

Günümüzde akıllı şehir uygulamaları, şehirlerin daha verimli yönetilebilmesi amacıyla yaygın olarak kullanılmaktadır. Özellikle IoT (Internet of Things) teknolojileri sayesinde çevresel veriler gerçek zamanlı olarak toplanabilmekte ve analiz edilebilmektedir. Bu veriler sayesinde sıcaklık, trafik yoğunluğu, hava kalitesi ve enerji tüketimi gibi birçok bilgi merkezi sistemler üzerinden takip edilebilmektedir.

Bu proje kapsamında, akıllı şehirlerde kullanılabilecek IoT tabanlı bir sıcaklık izleme sistemi geliştirilmiştir. Sistem üzerinde farklı bölgelerde bulunan sıcaklık sensörlerinden veri toplanmış, bu veriler AWS bulut servislerine gönderilmiş ve gerçek zamanlı olarak görselleştirilmiştir.

Projede Python programlama dili kullanılarak sanal sıcaklık sensörleri geliştirilmiştir. Sensörlerden elde edilen sıcaklık verileri MQTT protokolü ile AWS IoT Core servisine aktarılmıştır. Daha sonra AWS Lambda kullanılarak işlenen veriler DynamoDB veritabanına kaydedilmiş ve Streamlit kullanılarak geliştirilen dashboard üzerinden kullanıcıya sunulmuştur.

Bu proje sayesinde IoT sistemlerinin bulut bilişim servisleri ile nasıl entegre çalıştığı öğrenilmiş ve gerçek zamanlı veri işleme mimarisi geliştirilmiştir.

2. Proje Tanımı

Bu proje, akıllı şehirlerde sıcaklık verilerinin gerçek zamanlı olarak takip edilmesini sağlayan IoT tabanlı bir sistemdir. Sistem, farklı bölgelerde bulunan sıcaklık sensörlerinden veri toplamakta ve bu verileri AWS bulut servisleri üzerinde işlemektedir.

Projede fiziksel sensör yerine Python ile geliştirilmiş sanal sensörler kullanılmıştır. Bu sensörler belirli aralıklarla rastgele sıcaklık verileri üretmekte ve MQTT protokolü üzerinden AWS IoT Core servisine göndermektedir.

AWS IoT Core üzerinden gelen veriler bir IoT Rule yardımıyla AWS Lambda fonksiyonuna aktarılmaktadır. Lambda fonksiyonu gelen verileri işleyerek DynamoDB veritabanına kaydetmektedir. Son olarak Streamlit kullanılarak geliştirilen dashboard üzerinden:

- Sıcaklık değerleri görüntülenebilmekte,
- Grafikler oluşturulabilmekte,
- Ortalama sıcaklık hesaplanabilmekte,
- Kritik sıcaklık değerleri takip edilebilmektedir.

Sistem tamamen gerçek zamanlı çalışmakta ve dashboard belirli aralıklarla otomatik olarak güncellenmektedir.

3. Kullanılan Teknolojiler

Bu proje kapsamında IoT, bulut bilişim ve veri görselleştirme alanlarında yaygın olarak kullanılan teknolojiler tercih edilmiştir.

Python

Projenin temel geliştirme dili olarak Python kullanılmıştır. Python dili;

- MQTT bağlantıları,
- AWS servis entegrasyonları,
- Veri işleme,
- Dashboard geliştirme süreçlerinde kullanılmıştır.

MQTT Protokolü

MQTT (Message Queuing Telemetry Transport), IoT sistemlerinde yaygın olarak kullanılan hafif bir mesajlaşma protokolüdür.

Bu projede sıcaklık sensörlerinden elde edilen veriler MQTT kullanılarak AWS IoT Core servisine gönderilmiştir.

AWS IoT Core

Bu projede AWS IoT Core:

- Sensör bağlantılarını yönetmek,
- MQTT mesajlarını almak,
- Gelen verileri Lambda fonksiyonuna aktarmak amacıyla kullanılmıştır.

IoT cihaz güvenliği için sertifika tabanlı doğrulama mekanizması uygulanmıştır.

3. Kullanılan Teknolojiler

AWS Lambda

Bu projede Lambda fonksiyonu:

- AWS IoT Core'dan gelen sıcaklık verilerini almak,
- Verileri işlemek,
- DynamoDB veritabanına kaydetmek

amacıyla kullanılmıştır.

Lambda sayesinde herhangi bir sunucu yönetimi yapılmadan backend işlemleri gerçekleştirilebilmiştir.

DynamoDB

Bu projede sıcaklık sensörlerinden gelen verilerin saklanması amacıyla kullanılmıştır.

Veriler aşağıdaki alanlarla birlikte kaydedilmiştir:

- device_id
- location
- temperature
- timestamp

Streamlit

Bu projede:

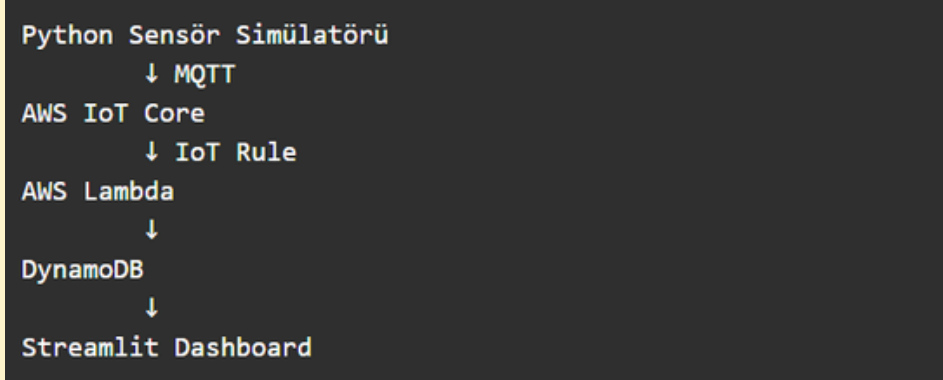
- Gerçek zamanlı sıcaklık grafikleri,
- Veri tabloları,
- Ortalama sıcaklık hesaplamaları,
- Kritik sıcaklık uyarıları

oluşturmak amacıyla kullanılmıştır.

4. SİSTEM MİMARİSİ

Bu projede IoT tabanlı istemci-sunucu mimarisi kullanılmıştır.

Sistemin genel çalışma yapısı aşağıdaki gibidir:



Sensör Katmanı

Sensör katmanında Python ile geliştirilmiş sanal sıcaklık sensörleri bulunmaktadır. Bu sensörler:

- Rastgele sıcaklık üretmekte,
- Konum bilgisi eklemekte,
- Timestamp oluşturmaktadır.

Üretilen veriler MQTT ile AWS IoT Core'a gönderilmektedir.

AWS IoT Core Katmanı

Bu katman IoT cihazları ile AWS servisleri arasındaki bağlantıyı sağlamaktadır.

AWS IoT Core:

- MQTT bağlantılarını kabul etmekte,
- Sensör verilerini almakta,
- IoT Rule yardımıyla Lambda'ya yönlendirmektedir.

4. SİSTEM MİMARİSİ

Lambda Katmanı

Lambda katmanı sistemin backend kısmını oluşturmaktadır.

Bu katmanda:

- Gelen veriler işlenmekte,
- Veri formatı kontrol edilmekte,
- DynamoDB'ye kayıt işlemleri yapılmaktadır.

Veritabanı Katmanı

Bu katmanda DynamoDB kullanılmaktadır. Sıcaklık verileri NoSQL yapısında saklanmaktadır. Her veri kaydı aşağıdaki alanları içermektedir:

- device_id
- timestamp
- location
- temperature

Dashboard Katmanı

Dashboard katmanı kullanıcı arayüzünü oluşturmaktadır.

Bu katmanda:

- Sıcaklık grafikleri,
- Veri tabloları,
- Ortalama sıcaklık değerleri,
- Alarm sistemleri

kullanıcıya sunulmaktadır.

5. Veri Akış Yapısı

5.1 MQTT Veri Gönderimi

Python sensör simülatörü belirli aralıklarla sıcaklık verisi üretmektedir.

Örnek veri yapısı aşağıdaki gibidir:

```
{  
  "device_id": "sensor_1",  
  "location": "Park",  
  "temperature": 28.5,  
  "timestamp": "2026-05-09T14:30:00"  
}
```

5.2 Lambda Veri İşleme Süreci

AWS IoT Core üzerinde oluşturulan IoT Rule sayesinde gelen veriler otomatik olarak Lambda fonksiyonuna aktarılmaktadır.

Lambda fonksiyonu:

- Veriyi almakta,
- JSON formatını işlemekte,
- DynamoDB'ye kayıt oluşturmaktadır.

5.3 DynamoDB Veri Saklama Yapısı

DynamoDB tablosunda:

- device_id → Partition Key
- timestamp → Sort Key

olarak belirlenmiştir.

Bu yapı sayesinde sensör verileri düzenli bir şekilde saklanabilmektedir.

6. Sensör Simülasyonu

6.1 Python Sensör Yapısı

Sensör sistemi Python dili kullanılarak geliştirilmiştir.

Sistem belirli aralıklarla otomatik veri üretmektedir.

6.2 Rastgele Veri Üretimi

Sensörler 20°C ile 40°C arasında rastgele sıcaklık üretmektedir.

Ayrıca aşağıdaki konumlar rastgele seçilmektedir:

- Merkez
- Park
- Üniversite
- Sanayi
- Otobüs Durağı

6.3 MQTT Publish İşlemleri

Üretilen veriler MQTT publish işlemi ile AWS IoT Core'a gönderilmektedir.

Bağlantı güvenliği için TLS sertifikaları kullanılmıştır.

7. Dashboard Geliştirme

7.1 Streamlit Yapısı

Dashboard geliştirme sürecinde Streamlit kullanılmıştır.

Dashboard üzerinden:

- Sensör verileri görüntülenebilmekte,
- Grafikler oluşturulabilmekte,
- Sıcaklık analizleri yapılabilmektedir.

7.2 Grafik Oluşturma

Sıcaklık değişimlerini göstermek amacıyla Plotly kullanılarak çizgi grafikleri oluşturulmuştur.

Bu grafikler sayesinde kullanıcılar sıcaklık değişimlerini gerçek zamanlı olarak takip edebilmektedir.

7.3 Gerçek Zamanlı Güncelleme

Dashboard sistemi belirli aralıklarla otomatik olarak yenilenmektedir.

Bu sayede yeni gelen sensör verileri kullanıcıya anlık olarak gösterilmektedir.

7.4 Alarm Sistemi

Dashboard içerisinde kritik sıcaklık değerlerini göstermek amacıyla alarm sistemi geliştirilmiştir.

35°C üzerindeki sıcaklık değerleri kritik olarak değerlendirilmiş ve kullanıcıya uyarı mesajı gösterilmiştir.

7. Dashboard Geliştirme



Akıllı Şehir Sıcaklık İzleme Sistemi

IoT sensörlerinden gelen sıcaklık verilerinin gerçek zamanlı izlenmesi.

Son Sıcaklık

23.02 °C

Ortalama

30.62 °C

En Yüksek

39.59 °C

En Düşük

20.96 °C



Son Veriler

	device_id	location	temperature	timestamp
27	sensor_1	Üniversite	38.03	2026-05-09 14:35:08
18	sensor_2	OtobusDuragi	25.62	2026-05-09 14:35:13
9	sensor_4	OtobusDuragi	38.01	2026-05-09 14:35:18
38	sensor_5	Merkez	22.04	2026-05-09 14:35:23
39	sensor_5	Sanayi	35.63	2026-05-09 14:35:28
10	sensor_4	Üniversite	37.81	2026-05-09 14:35:33
28	sensor_1	Üniversite	24.37	2026-05-09 14:35:38
4	sensor_3	Merkez	28.97	2026-05-09 14:35:43
40	sensor_5	Sanayi	33.59	2026-05-09 14:35:48
29	sensor_1	Sanayi	34.92	2026-05-09 14:35:53

8. AWS Üzerinde Kurulum

8.1 IoT Thing Oluşturma

AWS IoT Core üzerinde sanal sensör cihazı oluşturulmuştur.

Thing oluşturma sürecinde:

- Sertifikalar oluşturulmuş,
- Policy tanımlanmış,
- MQTT erişim izinleri verilmiştir.

8.2 Sertifika Yönetimi

AWS IoT Core bağlantılarında güvenlik amacıyla:

- Device certificate
- Private key
- Root CA

kullanılmıştır.

Bu sertifikalar sayesinde sensör bağlantıları güvenli hale getirilmiştir.

8.3 IoT Rule Oluşturma

AWS IoT Core üzerinde oluşturulan rule sayesinde gelen MQTT mesajları otomatik olarak Lambda fonksiyonuna yönlendirilmiştir.

Kullanılan SQL yapısı:

```
SELECT * FROM 'smartcity/temperature'
```

şeklindedir.

8.4 Lambda Yetkilendirme

Lambda fonksiyonuna DynamoDB erişimi sağlamak amacıyla IAM üzerinden gerekli izinler verilmiştir.

8. AWS Üzerinde Kurulum

8.5 DynamoDB Kurulumu

TemperatureData isimli DynamoDB tablosu oluşturulmuştur.

Tablo yapısında:

- device_id → Partition Key
- timestamp → Sort Key

olarak yapılandırılmıştır.

9. Sonuç

Bu proje kapsamında IoT tabanlı gerçek zamanlı sıcaklık izleme sistemi geliştirilmiştir.

Projede AWS bulut servisleri ile IoT sistemlerinin birlikte nasıl çalıştığı uygulamalı olarak öğrenilmiştir.

Geliştirilen sistem sayesinde:

- Sensör verileri gerçek zamanlı olarak toplanmış,
- AWS bulut servisleri üzerinde işlenmiş,
- Veritabanına kaydedilmiş,
- Dashboard üzerinden kullanıcıya sunulmuştur.

Bu proje, akıllı şehir sistemleri için temel bir örnek oluşturmakta olup ilerleyen süreçlerde:

- Gerçek fiziksel sensörler,
- Mobil uygulamalar,
- Harita sistemleri,
- Yapay zeka destekli analizler

ile daha gelişmiş hale getirilebilir.

Proje sayesinde IoT, bulut bilişim, serverless mimari ve gerçek zamanlı veri işleme konularında önemli deneyimler kazanılmıştır.